RELATÓRIO TÉCNICO — FILASMART

Avaliação A2 — Programação para Dispositivos Móveis

Integrante: Marcos Gabryel da Silva Moreira | Turma: Matutino | Semestre: 4º semestre | Professor: Genivaldo Carlos Da Silva

1. Contexto, problema e objetivo

Filas em secretarias, bibliotecas, laboratórios e setores acadêmicos causam incerteza e deslocamentos desnecessários. O FilaSmart é um protótipo mobile para organizar o ingresso do estudante em filas, guardar a senha atual no aparelho e apresentar atualizações de forma compreensível.

O objetivo é aplicar React Native, Expo e TypeScript em um projeto integrador com componentes, estado, hooks, formulário, navegação, consumo REST/JSON, persistência local, recurso do dispositivo e diferenças multiplataforma.

2. Funcionalidades implementadas

- Lista de quatro serviços acadêmicos com FlatList.
- Entrada na fila por formulário com TextInput e validação de nome obrigatório.
- Geração de senha com posição inicial calculada conforme o serviço escolhido.
- Acompanhamento da senha persistida, com atualização simulada da posição.
- Painel de atualizações consumindo JSON por API REST pública.
- Preferências para solicitar permissão de notificações locais.
- Feedback háptico em botões de ação.

3. Arquitetura e estrutura de pastas

As rotas ficam em src/app. Componentes de apresentação reutilizáveis ficam em src/components. A regra de criação de senha e a validação ficam em src/features/fila. O hook src/hooks/useSenhaAtual concentra estado, restauração e persistência. Os adaptadores de API, AsyncStorage e notificações ficam em src/services.

4. Telas e fluxo de navegação

1. Início: exibe serviços, senha ativa e atualizações REST.
2. Entrada na fila: recebe o serviço selecionado, valida os dados e salva a senha.
3. Acompanhamento: apresenta posição, status e ação de atualização.
4. Preferências: explica e solicita a permissão de notificações locais.

5. Componentes, props, estado e hooks

CartaoServico recebe o serviço e a ação por props. BotaoAcao recebe rótulo, ação e variante, fornecendo retorno tátil sem conter a regra de negócio. As telas usam useState para campos, carregamento, erros e respostas da API. O hook useSenhaAtual usa useEffect para restaurar a senha persistida quando a tela é aberta.

6. Formulário e validações

A tela Entrada na fila possui TextInput de nome completo. A função validarEntradaNaFila impede a criação da senha quando o nome está vazio ou quando não existe serviço selecionado. As mensagens são exibidas diretamente junto ao campo para orientar o estudante.

7. API REST e dados JSON

A aplicação consulta GET https://jsonplaceholder.typicode.com/todos?_limit=4. O JSON retornado contém id, title e completed. Os dados são processados em fila-api.ts e apresentados como atualizações acadêmicas. A API é usada somente como fonte pública didática; não representa uma fila institucional real.

8. Estados de interface

O painel REST trata carregamento com indicador visual, sucesso com lista de atualizações, ausência de dados com mensagem específica e erro de comunicação com orientação para verificar a conexão. A senha local também trata carregamento durante a restauração e estado vazio quando o aluno ainda não entrou em uma fila.

9. Persistência local

A senha atual é serializada em JSON e armazenada com @react-native-async-storage/async-storage na chave

@filasmart/senha-atual. O App não acessa o armazenamento diretamente; o serviço fila-storage encapsula essa dependência e o hook usa a interface exposta.

10. Recurso Expo, permissões e multiplataforma

expo-haptics gera resposta tátil em ações relevantes. expo-notifications solicita permissão de forma explícita na tela Preferências, cria canal Android e dispara um aviso local após uma atualização de posição. O projeto usa Platform.OS e Platform.select para informar a plataforma e ajustar feedback/elevação. O código das telas e regras é compartilhado entre Android, iOS e web quando possível.

11. Testes e validações

- 7 testes automatizados aprovados: domínio da senha, validação, persistência, API e botão háptico.
- Verificação TypeScript com npx tsc --noEmit aprovada.
- Lint com npm run lint aprovado.
- Compatibilidade de dependências com npx expo install --check aprovada.
- Bundles exportados para web, Android e iOS com Expo.

12. Builds, versionamento e assinatura

Versão do aplicativo: 1.0.0. Identificador Android/iOS: com.marcosgabryel.filasmart. O eas.json define Development Build, APK preview e AAB production. As credenciais Android são gerenciadas remotamente pelo EAS e não foram incluídas no projeto, relatório ou ZIP.

Perfil | Build EAS | Situação no momento do relatório
Development | https://expo.dev/accounts/moreiragabryel/projects/filasmart/builds/f7bb9bbd-d346-4547-9b8d-b6ffdd27ec39 | Em fila
Preview APK | https://expo.dev/accounts/moreiragabryel/projects/filasmart/builds/26675ef7-f366-4d44-80cc-dd55dfb5ff60 | Em processamento
Production AAB | https://expo.dev/accounts/moreiragabryel/projects/filasmart/builds/ef1bafbc-3953-4c65-9758-e0eb34ae5403 | Em processamento

13. Limitações e ações pendentes do integrante

- Instalar o APK e o Development Build em um Android físico quando estiver disponível.
- Aceitar a permissão de notificações e registrar uma evidência do aviso de mudança de posição.
- Registrar modelo do aparelho, versão Android, data e resultado no CHECKLIST_TESTES.md.
- Se houver iPhone disponível, validar navegação, persistência e solicitação de permissão no iOS.
- Atualizar este relatório com o status final e os links efetivos dos três builds EAS ao término do processamento.

14. Problemas encontrados e soluções

O template inicial continha componentes e rotas não relacionados ao problema. Eles foram removidos para manter a navegação focada no domínio. O AsyncStorage exigiu mock no ambiente Jest, pois o módulo nativo não existe durante testes unitários. Para Development Build, foi necessário instalar expo-dev-client antes da solicitação no EAS.

15. Conclusão

O FilaSmart entrega um protótipo funcional e organizado para o cenário de filas acadêmicas, atendendo aos requisitos técnicos centrais da avaliação. A validação física de notificações e instalação depende da etapa final com Android real, que deve ser registrada pelo integrante para complementar as evidências.